ChangeWatch — Complete Web Dev Interview Guide

Prepared for VinnovateIT Club Interview (Web Development Subdomain) • First-Year CSE, VIT Vellore

> **How to use this guide:** This document is designed specifically for you as a 1st-year CSE fresher. Every single concept is explained from first principles with real-life analogies. All code references, numbers, and features come directly from the real ChangeWatch codebase in this repository. Read it aloud before your interview.

1. The Project in 60 Seconds

The Problem: When you want to track information online—like an out-of-stock sneaker on Amazon, a change in college exam notices on VTOP, or a price drop on a flight—you have to open your browser and refresh the page manually 50 times a day. Existing tools often trigger false alarms because website clocks, live view counters, and rotating ads constantly change even when the real content didn't.

Who Would Use It: Students tracking course registration portals, shoppers waiting for price drops, researchers tracking government updates, and developers monitoring their company's competitor websites.

What ChangeWatch Does: ChangeWatch is a full-stack automated web application that monitors any webpage in the background. It takes a snapshot of the webpage using an invisible browser robot, strips away random noise (like clocks and ads), compares the clean text and screenshots with the previous version, and instantly sends an alert to Discord, Telegram, or Email only when meaningful content actually changes.

> 🍽️ **Real-Life Analogy (The Restaurant):**
> • **Frontend (React):** The dining hall with tables, menus, and buttons where the customer sits.
> • **Backend (Express):** The kitchen where chefs cook and process orders.
> • **API (REST):** The waiter carrying orders from the dining table to the kitchen and bringing back meals.
> • **Database (SQLite):** The locked pantry storage room where recipes, ingredients, and past receipts are kept safely.
> • **Headless Browser (Playwright):** An invisible robot assistant sent out to an external grocery store to take a photo of the price tag and bring it back to our kitchen.

2. Pitches You Can Say Aloud

30-Second Elevator Pitch (Memorize This!)

> "Hi! I built ChangeWatch, an automated website change detection platform. It solves the pain of manually refreshing pages for exam circulars or price drops. It uses a React frontend, an Express backend, and a headless Playwright browser to snapshot pages, filter out useless noise like live clocks, compute exact word-level and visual diffs, and notify users via Discord, Telegram, or Email. What makes it unique is that even if 200 users track the same link, it only visits the site once to protect server resources."

2-Minute Detailed Pitch

> "Good morning! During my first semester, I noticed how tedious it is to constantly refresh portals for grade updates, hostel bookings, or tech sales. So I built ChangeWatch.
>
> On the frontend, I used React 19, TypeScript, and Tailwind CSS. Users can paste any URL, pick an interval, and even use an interactive Visual Selector Picker where they click on a specific element—like just a price tag—to monitor.
>
> On the backend, I built an Express 5 REST API connected to an SQLite database. What makes web monitoring hard is 'false alarms'—ads and live clocks change every second. To solve this, I built a noise normalization engine using Cheerio that strips dynamic timestamps and tokens. For change detection, it computes Myers word-by-word text diffs and pixel-by-pixel image diffs with Pixelmatch.
>
> For scalability, I implemented a shared-fetch queue: if 200 users monitor the same page, the backend bundles them into a single browser fetch, giving a 99.5% reduction in network overhead. For security, it has full SSRF protection so nobody can abuse the crawler to attack private networks. Building this taught me full-stack architecture, asynchronous job workers, and web security."

3. How the Website Works Step by Step

Here is what happens behind the scenes from the moment you click "Add Monitor" to receiving an alert:

```
[User Browser]
      │
      │ 1. Submits Form (URL: "https://example.com", CSS: ".price")
      ▼
[React Frontend (Client)]
      │
      │ 2. Sends HTTP POST /api/subscriptions with JSON payload + CSRF cookie
      ▼
[Express Server (Backend)]
      │
      │ 3. Validates URL (SSRF check: blocks localhost/private IPs)
      │ 4. Stores subscription record in SQLite DB
      ▼
[SQLite Database]
      │
      │ 5. Background Scheduler polls every 30s: "Which URLs are due?"
      ▼
[Worker Pool (3 Workers)]
      │
      │ 6. Launches Playwright (Invisible Chromium) to visit webpage
      │ 7. Captures HTML DOM and full-page PNG screenshot
      │ 8. Cheerio extracts clean text & strips dynamic noise (clocks/ads)
      │ 9. Myers Diff compares new text against baseline version in DB
      ▼
[Change Detection Decision]
      ├── If No Change (or 503 outage / CAPTCHA): Do NOT alert. Keep baseline.
      └── If Real Change Detected:
            │
            ├─► 10. Save new Version 2 in SQLite DB
            ├─► 11. Enqueue alert into notification_outbox table
            └─► 12. Dispatcher sends message to Discord / Telegram / Email!
```

> **Key Architectural Flow:**
> 1. **Input:** User inputs URL in React UI -> Sent via `fetch()` REST API call.
> 2. **Storage:** Express writes record into `subscriptions` table in SQLite.
> 3. **Async Queue:** Background scheduler checks due dates, groups duplicate URLs, and hands jobs to workers.
> 4. **Capture & Compare:** Worker runs Playwright, extracts clean text via Cheerio, runs Myers Diff algorithm.
> 5. **Output:** React UI displays color-coded green/red diffs; webhooks notify user.

4. File and Folder Guide

ChangeWatch follows a clean separation of concerns. Here is what each directory does:

Folder / File	What It Does	Should I Open in Interview?
`src/client/App.tsx`	Main React shell: renders navigation bar, tab switching, and top banner.	**Yes** (Great starting point)
`src/client/pages/Dashboard.tsx`	Shows monitoring stats, list of tracked URLs, status badges, and action buttons.	**Yes** (Shows clean UI logic)
`src/client/pages/Compare.tsx`	Myers text diff viewer (green additions, red deletions) and visual image slider.	**Yes** (The "Wow" page!)
`src/client/pages/VisualPicker.tsx`	Renders remote website preview so user can click to select CSS selectors.	Optional
`src/server/index.ts`	Backend entry point: boots Express, opens SQLite DB, starts workers.	**Yes** (Shows server architecture)
`src/server/routes/subscriptions.ts`	REST API endpoints to create, list, and delete monitored websites.	**Yes** (Shows Express routing)
`src/server/engine/changeDetector.ts`	Core engine: runs Cheerio text extraction, noise filtering, and Myers diffing.	**Yes** (Shows algorithm logic)
`src/server/capture/routeValidation.ts`	SSRF security shield: resolves DNS and blocks private/loopback IP addresses.	Great for security questions
`src/server/db/schema.sql`	Database schema: tables for users, targets, versions, diffs, and notifications.	Good to show DB design

5. Tech Stack Explained

Everything used in ChangeWatch, why it was chosen, and its simple alternative:

Technology	What It Is	Why It Was Used Here	Simple Alternative
React 19	A JavaScript library for building user interfaces using reusable components.	Lets us update UI elements (like status badges) instantly without reloading the entire page.	Vanilla JavaScript or Vue.js
TypeScript	JavaScript with strict types (checks for bugs before running code).	Prevents silly typos (e.g. calling `user.namme` instead of `user.name`) and ensures clean contracts.	Plain JavaScript
Tailwind CSS	A utility-first CSS framework using class names directly in HTML.	Builds modern, responsive dark-mode layouts quickly without writing hundreds of custom CSS files.	Bootstrap or custom CSS
Vite	A super-fast modern frontend build tool and dev server.	Bundles React code with lightning-fast hot module reloading during development.	Webpack or Create React App
Node.js & Express 5	A server runtime (Node) and minimalist web framework (Express) for building REST APIs.	Standard, lightweight, and easily handles asynchronous background tasks like scheduling and web crawling.	Python FastAPI or Flask
SQLite (better-sqlite3)	A fast, serverless, file-based relational database.	Zero setup, lightning fast with Write-Ahead Logging (WAL), stores all state in a single local `.db` file.	PostgreSQL or MySQL
Playwright	A headless browser automation tool maintained by Microsoft.	Renders modern JavaScript-heavy websites (SPAs) exactly like a human user's Google Chrome.	Puppeteer or Selenium
Cheerio	A fast HTML parser implementing jQuery syntax on the server.	Extracts text from HTML tags (`<p>`, `<h1>`) in milliseconds without loading heavy browser engines.	jsdom or regex parsing
Myers Diff (diff)	An algorithm for computing the shortest sequence of additions and deletions between texts.	Identifies the exact words or lines changed so users see clear green additions and red deletions.	Levenshtein distance
Pixelmatch & PNGjs	A pixel-by-pixel image comparison library for Node.js.	Compares page screenshots and highlights visual changes (e.g. banner updates) with an overlay.	Resemble.js

6. Beginner Glossary of Web Development Terms

- **DOM (Document Object Model):** The tree structure of HTML elements on a page. The browser turns HTML text into DOM objects that JavaScript can inspect, click, or modify.
- **Component:** A self-contained, reusable building block in React (like a Navbar, Button, or Card) that returns JSX (HTML-like syntax).
- **Props (Properties):** Data passed from a parent React component to a child component (like passing a title string into a Card component).
- **State:** Internal data held inside a component that can change over time (e.g. `const [monitors, setMonitors] = useState([])`). When state changes, React re-renders the UI automatically.
- **Routing:** Determining what UI or backend response to serve based on the URL path (e.g. visiting `/settings` vs `/compare`).
- **API (Application Programming Interface):** A set of rules and URLs that allows the React frontend to ask the Express backend for data.
- **REST (Representational State Transfer):** An API architectural style that uses standard HTTP methods (`GET`, `POST`, `PATCH`, `DELETE`) on resource URLs (e.g. `POST /api/subscriptions`).
- **JSON (JavaScript Object Notation):** The standard text format for exchanging data between client and server (looks like `{"url": "https://...", "interval": 15}`).
- **HTTP Methods:**
  - `GET`: Retrieve data (read-only, no server changes).
  - `POST`: Create a new resource (e.g. add a new website to track).
  - `PATCH / PUT`: Update an existing resource.
  - `DELETE`: Remove a resource.
- **CSRF (Cross-Site Request Forgery):** A web attack where a malicious site tricks your browser into making unwanted actions on another site. We prevent this using `X-CSRF-Token` headers.
- **SSRF (Server-Side Request Forgery):** An attack where a hacker asks our server crawler to visit internal private addresses (like `http://localhost:8080` or AWS cloud metadata). We prevent this by checking IP ranges and blocking private addresses.
- **Responsive Design:** Designing websites that automatically adapt their layout to look great on phones, tablets, and desktop monitors (achieved via Tailwind responsive prefixes like `sm:`, `md:`, `lg:`).

7. 25 Most Likely Interview Questions with Answers

Group A: Basic Questions

Q1: Can you introduce your project and what problem it solves?

> "ChangeWatch is an automated website monitoring tool. It tracks public web pages in the background, extracts meaningful content while filtering out dynamic noise like clocks, and notifies users via Discord or Email only when true changes happen."

Explanation: Focus on the real-world utility—preventing manual page refreshing and eliminating false alarms from ads or clocks.

Likely Follow-up: Why not just use existing tools like Visualping?
Answer: "Commercial tools have strict paywalls, slow intervals, and don't offer shared-fetch deduplication where 200 users tracking one page only consumes a single network request."

Q2: What was your specific role in building this project?

> "I architected the full-stack system, designed the React UI components, built the Express REST endpoints, designed the SQLite database schema, and implemented the noise-filtering logic to stop false alerts."

Explanation: Frame your contributions around understanding the flow, connecting frontend with backend, and deciding the data structures.

Likely Follow-up: What was the most fun part to build?
Answer: "Building the Myers Diff viewer in React because seeing the exact green additions and red deletions side-by-side felt like building our own mini-GitHub!"

Q3: Why did you pick React instead of plain HTML/CSS/JavaScript?

> "With plain JavaScript, updating the UI when data changes requires manually manipulating the DOM with `document.getElementById`, which gets messy. React uses declarative state: whenever my monitor list changes in state, the UI updates automatically and efficiently."

Explanation: Interviewers want to see that you understand the concept of state management and reusable components.

Likely Follow-up: What is the Virtual DOM in React?
Answer: "It's a lightweight in-memory copy of the real DOM. When state changes, React compares the Virtual DOM with the previous snapshot (called 'diffing') and only touches the exact browser elements that changed."

Group B: Frontend & UI Questions

Q4: How did you design the layout and make it responsive?

> "I used Tailwind CSS with CSS Flexbox and Grid. For example, the Dashboard stats cards use a responsive grid: 1 column on mobile screens (`grid-cols-1`) and 4 columns on desktop (`sm:grid-cols-2 lg:grid-cols-4`)."

Explanation: Demonstrates knowledge of modern mobile-first styling practices.

Likely Follow-up: What is the difference between CSS Flexbox and CSS Grid?
Answer: "Flexbox is one-dimensional (good for laying items out in a single row or column, like a navigation bar), while Grid is two-dimensional (great for rows and columns simultaneously, like a dashboard of cards)."

Q5: How does the React frontend communicate with the backend?

> "The frontend uses the native JavaScript `fetch()` API to make asynchronous HTTP requests to our Express endpoints (like `/api/subscriptions`). We manage loading states with React's `useState` and trigger fetches on mount using `useEffect` ."

Explanation: Highlights core asynchronous programming and lifecycle understanding in React.

Likely Follow-up: What is `useEffect` and when does it run?
Answer: " `useEffect` lets us perform side effects in components. With an empty dependency array `[]` , it runs once right after the component first mounts to the screen."

Q6: How does the Visual Selector Picker work on the frontend?

> "The user enters a URL, the backend captures an interactive DOM snapshot, and renders it inside an isolated preview container. When the user hovers or clicks an element (like an `h1` or `.price` tag), JavaScript calculates its unique CSS selector path and populates the form input."

Explanation: Explains how a complex feature translates into simple DOM event handling.

Likely Follow-up: Why is monitoring a specific selector better than the whole page?
Answer: "Monitoring only a CSS selector isolates just the relevant text (like a price tag) and ignores changes happening elsewhere in the header, sidebar, or footer."

Q7: What is the difference between Props and State in React?

> "Props are read-only inputs passed from a parent component to a child, like passing a monitor object into a card. State is local, mutable memory owned by the component itself, like whether a modal dialog is currently open or closed."

Explanation: A classic beginner web dev interview question.

Likely Follow-up: Can a child component modify its props?
Answer: "No, props are immutable in React. If a child needs to change parent data, the parent must pass down a callback function as a prop."

Group C: Backend & Data Questions

Q8: Why did you choose SQLite over MongoDB or PostgreSQL?

> "SQLite is an embedded, file-based relational database. It requires zero external daemon servers or Docker containers to run, making development and testing extremely fast. With WAL (Write-Ahead Logging) enabled, it easily handles concurrent reads and background worker writes."

Explanation: Shows practical engineering reasoning rather than just chasing trendy tech.

Likely Follow-up: What is WAL mode in SQLite?
Answer: "WAL stands for Write-Ahead Logging. Instead of locking the whole database file on every write, modifications are appended to a separate log file, allowing readers to read simultaneously while a writer writes."

Q9: What is the database schema structure in your project?

> "We have relational tables with foreign keys: `users` for accounts, `targets` for canonical URLs, `subscriptions` linking a user to a target with interval settings, `check_jobs` for the queue, `monitor_versions` storing clean text and screenshots, and `notification_outbox` for pending alerts."

Explanation: Demonstrates relational schema normalization knowledge.

Likely Follow-up: Why separate `targets` from `subscriptions`?
Answer: "That's how our shared-fetch deduplication works! Multiple users can subscribe to the same canonical target URL, but the worker only executes one network fetch for that target."

Q10: What are RESTful APIs and how are they implemented in Express?

> "RESTful APIs organize endpoints around resources using standard HTTP verbs. In our Express `subscriptions.ts` router, `GET /api/subscriptions` fetches the list, `POST` creates a new monitor, and `DELETE /api/subscriptions/:id` removes one using URL parameters."

Explanation: Direct, clear explanation of standard backend API routing.

Likely Follow-up: What status codes does your API return?
Answer: "`200 OK` for successful queries, `201 Created` on new records, `400 Bad Request` on invalid schemas, `401 Unauthorized` for missing auth, `403 Forbidden` for SSRF blocks, and `404 Not Found`."

Q11: How do you prevent website outages (like 503 errors) from triggering false change alerts?

> "If a website returns an HTTP 500, 502, or 503 error, or shows a Cloudflare CAPTCHA challenge, our change detector marks the check as an outage instead of a content change. The baseline version is left untouched so users don't get alerted that the entire page disappeared."

Explanation: Explains real-world fault tolerance in web scraping.

Likely Follow-up: When does the user get alerted for an outage?
Answer: "Only after 3 consecutive failures. If it recovers, the counter resets to 0 and sends a 'RECOVERED' event."

Group D: Code-Level Explanations (5 Key Code Snippets)

Q12: Can you explain how the Myers Diff algorithm works in your project?

> "In `src/server/engine/changeDetector.ts`, we use the `diff` library which implements the Myers algorithm. It compares two strings and finds the shortest edit script—marking chunks as either unchanged, added (green), or removed (red). On the client, `DiffViewer.tsx` maps over these chunks to render the highlighted view."

Line-by-Line Breakdown:

1. `const changes = diffWords(oldText, newText);` -> Computes token differences.
2. `part.added ? '+ ' : part.removed ? '- ' : ' '` -> Formats git-style diff markers.
3. `hasMeaningfulDiff = addedCount > 0 || removedCount > 0;` -> Checks if true change occurred.

Likely Follow-up: Why word diff instead of character diff?
Answer: "Character diffs are too granular and noisy for human reading; word diffs highlight exact word updates clearly."

Q13: How does your noise normalization filter work?

> "In `src/server/engine/noiseFilters.ts`, we use regex patterns and Cheerio to clean HTML. It removes `<script>`, `<style>`, and `<nav>` tags, then replaces dynamic timestamps (like '2 minutes ago' or '12:45 PM') and session tokens with static placeholder tokens like `[TIMESTAMP]`."

Line-by-Line Breakdown:

1. `$('script, style, noscript, iframe').remove();` -> Strips non-content elements.
2. `text.replace(/\b\d{1,2}:\d{2}(:\d{2})?\s*(AM|PM)?\b/gi, '[TIME]');` -> Replaces clocks.
3. `text.replace(/[a-f0-9]{32,64}/gi, '[TOKEN]');` -> Replaces random hex tokens.

Likely Follow-up: What if a user wants to monitor an actual clock?
Answer: "Noise filters are configurable checkboxes per monitor in the UI! Users can toggle timestamp filtering off if they wish."

Q14: How does your SSRF security validator work?

> "In `src/server/capture/routeValidation.ts`, before Playwright visits any URL, we resolve its hostname via DNS. We use `ipaddr.js` to verify that the resolved IP is a public unicast address, blocking loopback `127.0.0.1`, private LANs `192.168.x.x`, and cloud metadata `169.254.169.254`."

Line-by-Line Breakdown:

1. `const parsedIp = ipaddr.parse(resolvedAddress);` -> Parses IPv4 or IPv6 string.
2. `if (parsedIp.range() !== 'unicast') throw new Error('SSRF_BLOCKED');` -> Rejects private ranges.
3. We also intercept Playwright subresource requests to prevent DNS rebinding redirects.

Likely Follow-up: What is DNS rebinding?
Answer: "It's an attack where an initial DNS query returns a public IP to pass validation, but a second lookup moments later returns a private IP to access internal services. We mitigate this by checking at request time."

Q15: How does the background scheduler decide what jobs to run?

> "In `src/server/queue/scheduler.ts`, a `setInterval` ticks every 30 seconds. It executes a SQL query to find active subscriptions where `next_run_at <= now`. It groups them by target URL and inserts a single job into `check_jobs` with status 'QUEUED'."

Line-by-Line Breakdown:

1. `SELECT * FROM subscriptions WHERE status = 'ACTIVE' AND next_run_at <= datetime('now');`
2. `INSERT OR IGNORE INTO check_jobs (target_id, status) VALUES (?, 'QUEUED');`
3. Workers poll the queue, claim a job with atomic status update, and process the fetch.

Likely Follow-up: Why use a worker queue instead of fetching directly inside the HTTP request?
Answer: "Web crawling takes several seconds. Doing it synchronously in an HTTP request would freeze the user's browser; background workers keep the API instantaneous."

Q16: How does user authentication and session management work in your backend?

> "In `src/server/auth/`, passwords are encrypted using PBKDF2 with 600,000 hashing iterations and random salts. On login, we generate a cryptographically random session token stored in an `httpOnly` cookie, plus a CSRF token for protecting state-changing requests."

Line-by-Line Breakdown:

1. `crypto.pbkdf2(password, salt, 600000, 32, 'sha256', ...)` -> Secure password hashing.
2. `res.cookie('cw_session', token, { httpOnly: true, sameSite: 'lax' });` -> Sets secure cookie.
3. `verifyCsrf` middleware verifies `req.headers['x-csrf-token'] === req.session.csrfToken`.

Likely Follow-up: Why use `httpOnly` for cookies?
Answer: "It prevents malicious client-side JavaScript (XSS attacks) from reading or stealing the user's session token."

Group E: Deployment & Tools Questions

Q17: How is your project hosted and deployed?

> "The static frontend is hosted on GitHub Pages with an automated GitHub Actions deployment workflow. For local full-stack execution, you run `npm install` followed by `npm start`, which launches both the Express API and serves the production Vite client on port 3000."

Explanation: Clearly distinguishes the static showcase on GitHub Pages from the full local/cloud Node.js backend runtime.

Likely Follow-up: How does GitHub Pages work if there is no backend server running on it?
Answer: "We created a mock fetch adapter (`src/client/staticDemo.ts`) that intercepts client API calls on GitHub Pages to supply sample dashboard data and fixtures, with a clear banner explaining it's a static preview."

Q18: What is Git and how did you use it in this project?

> "Git is a distributed version control system that tracks every change made to our source code. We use Git to make atomic commits per feature (e.g. `fix(security): resolve SSRF checks`), collaborate on GitHub, and trigger automated CI/CD verification workflows on push."

Explanation: Shows practical understanding of software engineering workflows.

Likely Follow-up: What is the difference between Git and GitHub?
Answer: "Git is the command-line tool installed on my laptop that tracks code history; GitHub is a cloud-hosted platform for storing Git repositories and running CI/CD pipelines."

Q19: How do your CI/CD verification workflows work?

> "In `.github/workflows/ci.yml` , every push triggers an automated runner that executes `npm run verify` . This chains four strict gates: TypeScript typecheck (`tsc --noEmit`), ESLint with zero warnings allowed, 41 Vitest test suites (210 tests), and production Vite build."

Explanation: Impressive for a 1st year to speak knowledgeably about CI quality gates.

Likely Follow-up: Why enforce zero ESLint warnings?
Answer: "Warnings tend to accumulate into technical debt; enforcing zero warnings guarantees code cleanliness and catches unused variables or type issues immediately."

Group F: Challenges, Limitations, Bugs & Security

Q20: What was the hardest bug or challenge you faced, and how did you fix it?

> "The trickiest challenge was the Express 5 route wildcard change. In older Express 4, a catch-all route was written as `app.get('*')`. But in Express 5 with path-to-regexp v8, a bare asterisk throws a fatal syntax error! We diagnosed it and migrated to the proper named wildcard syntax: `app.get('/{*splat}')`."

Explanation: A real, concrete engineering bug that demonstrates debugging skill and modern library knowledge.

Likely Follow-up: How did you ensure it won't break again?
Answer: "We wrote a dedicated production boot smoke test (`scripts/prodSmoke.ts`) that boots the compiled server on an isolated port in CI and verifies all routes."

Q21: What are the current limitations of ChangeWatch?

> "Two main limitations: first, Playwright requires a Chromium binary which takes significant RAM, so running dozens of concurrent browsers requires careful worker pooling. Second, websites with aggressive Cloudflare Turnstile or CAPTCHAs will block automated scrapers, so we gracefully mark them as outages."

Explanation: Mature developers know the boundaries and real-world trade-offs of their architecture.

Likely Follow-up: How would you handle pages that don't need a heavy browser?
Answer: "We added an `http_dom` engine option that uses fast raw HTTP GET and Cheerio parsing, consuming 90% less memory for static pages."

Q22: What future features would you like to add?

> "I'd love to add Webhook integrations for Slack and Microsoft Teams, a mobile app using React Native, and AI-driven semantic summaries using Gemini to tell users 'Price dropped from ₹1000 to ₹800' in plain English."

Explanation: Shows vision and enthusiasm for expanding web technologies.

Likely Follow-up: Is the Gemini AI feature already in the code?
Answer: "Yes, we have an optional stretch module in `src/server/engine/geminiSummary.ts` that enriches diffs if an API key is provided, falling back gracefully to rule-based summaries if not."

Group G: Tricky & Pressure Questions

Q23: Did you build this entire project yourself, or did an AI write it?

> "I used Google's Antigravity AI coding assistant as an intelligent pair programmer. It helped me write boilerplates, refactor complex regex, and structure tests. However, I directed the architecture, made all design decisions, tested the code, and understand every single file and flow in this repo."

Explanation: Never lie. Clubs respect honesty, intellectual curiosity, and the ability to explain the underlying code thoroughly.

Likely Follow-up: If you used AI, prove you understand it: how does the subscription delete cascade work?
Answer: "In `src/server/maintenance/deleteFlow.ts`, when a subscription is deleted, we remove its foreign key records from `monitor_versions` and `check_runs`, unlink any screenshot PNG files on disk, and if no other users monitor that target, clean up the parent target too."

Q24: Why do you want to join the VinnovateIT Web Development subdomain?

> "VinnovateIT is known for building high-impact, production-grade applications, not just toy projects. I love building full-stack systems with React and Node.js, and I want to collaborate with seniors who can push my code quality, learn modern cloud architectures, and contribute actively to club hackathons."

Explanation: Tailored specifically to the ambition and culture of VinnovateIT at VIT.

Likely Follow-up: What can you contribute immediately?
Answer: "I am comfortable with modern TypeScript, React component design, Express REST APIs, and writing clean, tested code. I'm dependable, fast at learning, and ready to put in the hours."

Q25: Why should we select you over other 1st year applicants?

> "Most freshers only build basic static portfolios or todo-lists with vanilla HTML/CSS. I went several steps further by building a full-stack, event-driven web monitoring system with background job workers, database schemas, test automation, and security hardening. I have proven that I can take complex web concepts and turn them into working software."

Explanation: Confident, evidence-backed, and highlights initiative.

Likely Follow-up: How do you manage your time between club projects and academics?
Answer: "I follow structured time-blocking: weekdays for classes and fundamentals, and dedicated evening/weekend blocks for coding, git commits, and project milestones."

8. Honest Answer Guide: "Did You Use AI to Build This?"

> **The Golden Rule:** Never claim you coded 10,000 lines of complex TypeScript from scratch alone in semester 1. The interviewers are experienced seniors; they will immediately know. Instead, frame AI as a *modern developer multiplier* and prove you have mastery over the code.

The 3-Part Answer Framework:

1. **Acknowledge with Confidence:** "Yes, I used Antigravity (Google's AI coding assistant) as my pair programmer and mentor throughout this project."
2. **State What YOU Decided & Learned:** "I chose the tech stack (React, Express, SQLite), designed the user flow, specified the features (like shared-fetch deduplication and noise filtering), reviewed every diff, and debugged tricky edge cases like Express 5 wildcards and SSRF attacks."
3. **Offer Code Verification:** "I understand how every component and route works, and I'd be more than happy to open any file right now and explain the logic line by line."

9. Web Dev Fundamentals Every Fresher Must Know

Interviewers might ask general web concepts to test your basics. Here are quick, simple explanations:

- **How the Internet & Browser Load a Page:**
  1. You type `https://example.com` into your browser address bar.
  2. **DNS Lookup:** Your computer asks a DNS server (like phonebook directory) to translate the domain name into an IP address (e.g. `93.184.216.34`).
  3. **TCP/TLS Handshake:** Your browser opens a secure, encrypted connection to that server.
  4. **HTTP Request:** The browser sends an HTTP `GET /` request.
  5. **Rendering:** The server replies with HTML. The browser parses the HTML into a DOM tree, fetches linked CSS and JavaScript files, runs React, and paints the pixels on screen.
- **HTTP vs HTTPS:**
  • **HTTP:** HyperText Transfer Protocol. Data is sent in plain text across the network. If anyone intercepts it, they can read passwords and cookies.
  • **HTTPS:** HTTP Secure. Data is encrypted using TLS/SSL before transmission. Even if someone intercepts the packets, they only see scrambled gibberish.
- **Client vs Server:**
  • **Client:** The device and software running on the user's end (e.g. Chrome browser on your laptop). Handles user interaction and visual display.
  • **Server:** A computer located in a cloud data center that runs continuously. Listens for client requests, processes business logic, and queries the database.
- **Frontend vs Backend:**
  • **Frontend:** Everything the user sees and interacts with (buttons, forms, animations, styles). Built with HTML, CSS, JavaScript, React.
  • **Backend:** The invisible machinery behind the scenes (authentication, database queries, file storage, payment processing, background jobs). Built with Node.js, Express, Python.

10. 10-Minute Interview Time Allocation Plan

Time	Phase	What to Do & Say
Minute 0 – 2 (2 mins)	**Introduction & Pitch**	Greet the panel, give your 30-second or 2-minute elevator pitch with a warm, energetic smile. Mention you are a 1st-year CSE student passionate about full-stack web development.
Minute 2 – 5 (3 mins)	**Live Website Demo**	Follow the script from `DEMO_GUIDE.pdf`. Show the Dashboard, create a monitor, trigger a check, show the Myers diff page, and highlight the shared-fetch concept.
Minute 5 – 9 (4 mins)	**Technical & Code Questions**	Answer technical questions calmly. If asked to show code, open `Dashboard.tsx` or `changeDetector.ts` and explain the logic clearly.
Minute 9 – 10 (1 min)	**Closing & Passion**	State why you want to join VinnovateIT. Ask a thoughtful question (e.g. "What kind of web stacks does VinnovateIT plan to use in upcoming flagship hackathons?"). Thank them!

11. Last-Minute Cheat Sheet: 15 Facts to Remember

1. **Project Name:** ChangeWatch (Website Change Detector).
2. **Frontend Tech:** React 19, TypeScript, Tailwind CSS, Vite.
3. **Backend Tech:** Node.js (v22+), Express 5, Better-SQLite3 (WAL mode).
4. **Browser Automation:** Playwright (Headless Chromium).
5. **Text Parsing:** Cheerio (server-side fast HTML extraction).
6. **Diffing Algorithms:** Myers Diff (text/words) and Pixelmatch (screenshots).
7. **Port Numbers:** Port `3000` for main app; Port `4000` for local 16-fixture demo server.
8. **Demo Login Credentials:** `demo@changewatch.local` / `Password123!`
9. **Test Metrics:** 41 test files, 210 tests passing with 100% green status.
10. **Consolidation Ratio:** 20:1 and 200:1 shared-fetch deduplication (99.5% overhead reduction).
11. **Outage Protection:** 503 errors and CAPTCHAs leave baseline intact (no false alarms).
12. **Security Shield:** SSRF protection blocks `127.0.0.1`, `192.168.x.x`, and cloud metadata.
13. **Password Security:** PBKDF2 with 600,000 iterations + cryptographically random salts.
14. **Notification Channels:** Discord Webhooks, Telegram Bot API, and Email (Nodemailer).
15. **Live Link:** `https://vaibhavmittal29.github.io/changewatch/`

12. "Learn Tonight": 5 Concepts to Master Before Bed

1. React State & Props

Props are arguments passed into a component from outside (like function parameters); State is internal data held by the component that causes the UI to re-render when changed (using `useState`). Remember: Props flow down, events bubble up.

2. REST APIs & HTTP Verbs

A REST API lets the frontend request data using URLs. `GET` retrieves data without side effects; `POST` creates a new resource; `PATCH` updates part of a resource; `DELETE` removes it. Status 200 means success; 400 means client mistake; 500 means server crash.

3. Client-Side Rendering (CSR) vs Server-Side

In a React SPA (Single Page Application), the server sends a blank HTML shell and a JavaScript bundle. Your browser executes the JS bundle, builds the DOM in memory, and paints the UI without ever reloading the browser page.

4. Asynchronous JavaScript & Promises

Operations like network requests (`fetch()`) and database queries take time. `async/await` allows JavaScript to pause function execution non-blockingly until the data arrives, keeping the UI smooth and responsive.

5. SSRF (Server-Side Request Forgery)

A vulnerability where an attacker tricks a web crawler into fetching internal, private addresses (like `http://localhost:8080` or AWS metadata). We stop it by resolving DNS first and blocking all non-public IP addresses.